



# **DSP Synthesis and BRAM Optimization on UltraScale+ FPGA**

**Made by: Basem Hesham**



# Table of Contents

|     |                                                             |    |
|-----|-------------------------------------------------------------|----|
| 1   | DSP Block Synthesis and Timing Optimization .....           | 3  |
| 1.1 | Unit-Level Implementation & Resource Footprint .....        | 3  |
| 1.2 | Virtex UltraScale+ FPGA Feature Summary .....               | 5  |
| 1.3 | DSP48E2 Resource Mapping for 19-bit Arithmetic .....        | 5  |
| 1.4 | Timing Constraints & Critical Path Management .....         | 6  |
| 1.5 | Advanced Mathematical Optimization (SIMD Logic) .....       | 7  |
| 2   | BRAM Optimization and Memory Bandwidth Analysis .....       | 7  |
| 2.1 | Architectural Features of UltraScale+ BRAM .....            | 7  |
| 2.2 | Native Configurations and Parity Integration .....          | 8  |
| 2.3 | Design Trade-off Analysis: 19-bit Element Mapping .....     | 9  |
| 2.4 | Bandwidth and Throughput Conclusion.....                    | 9  |
| 2.5 | Storage Feasibility.....                                    | 10 |
| 2.6 | Computational Bandwidth Alignment: Stage 1 Case Study ..... | 10 |



## 1 DSP Block Synthesis and Timing Optimization

In this phase, we conducted a rigorous synthesis and timing analysis of the fundamental arithmetic units—multipliers and adders—to determine their hardware resource footprint and maximum performance on the Virtex UltraScale+ XCVU11P (xcvu11p-flga2577-3-e). The primary goal was to optimize for speed using dedicated silicon resources while maintaining a clean timing slack for the overall system integration.

### 1.1 Unit-Level Implementation & Resource Footprint

We utilized specific Xilinx IP cores to implement 19-bit fixed-point arithmetic. By targeting the **DSP48E2 slices** instead of LUT-based fabric, we achieved high-frequency stability suitable for our parallel architecture.

| Hardware Unit    | Vivado IP Name | Configuration       | Resource Usage | Best Period (Tclk) |
|------------------|----------------|---------------------|----------------|--------------------|
| Multiplier       | mult_gen_0     | 19-bit (Speed Opt.) | 2 DSP48 Slices | 2.5 ns (400 MHz)   |
| Adder/Subtractor | c_addsub_0     | 19-bit (Speed Opt.) | 1 DSP48 Slice  | 1.4 ns (714 MHz)   |

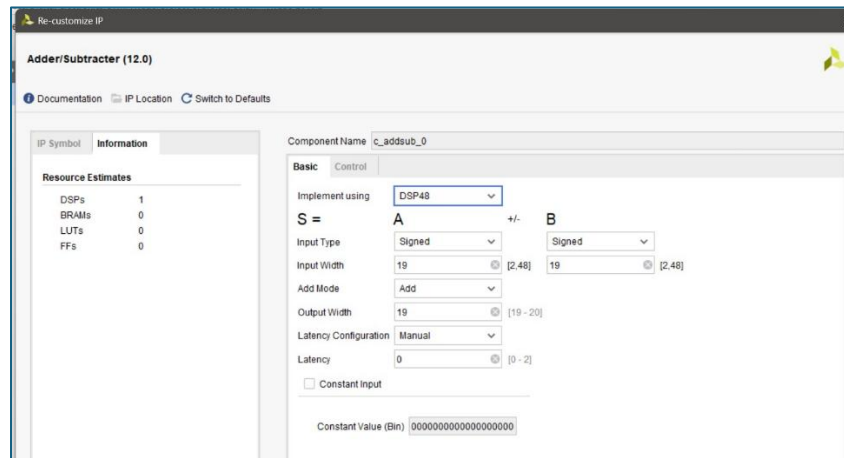


Figure 1: Adder/Subtractor IP Configuration

| Resource | Utilization | Available | Utilization % |
|----------|-------------|-----------|---------------|
| FF       | 57          | 2592000   | 0.00          |
| DSP      | 1           | 9216      | 0.01          |
| IO       | 59          | 448       | 13.17         |
| BUFG     | 1           | 1008      | 0.10          |

Figure 2: Adder/Subtractor IP Utilization

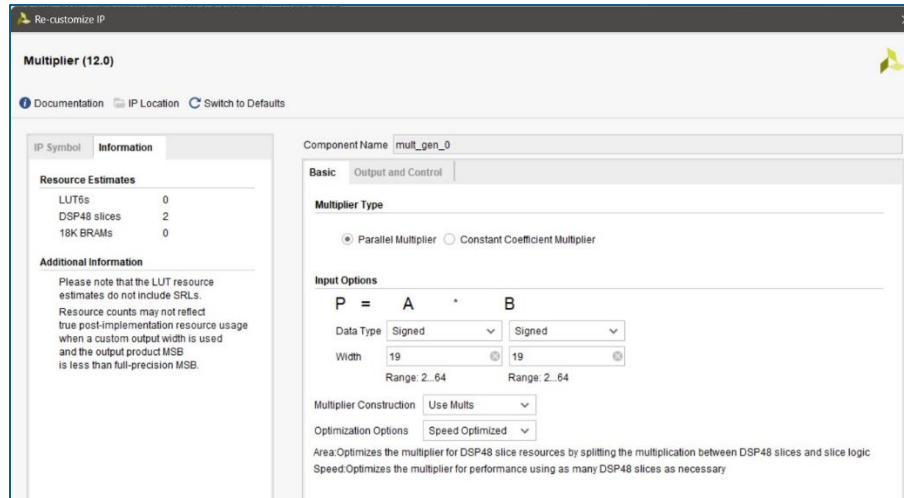


Figure 3: Multiplier IP Configuration

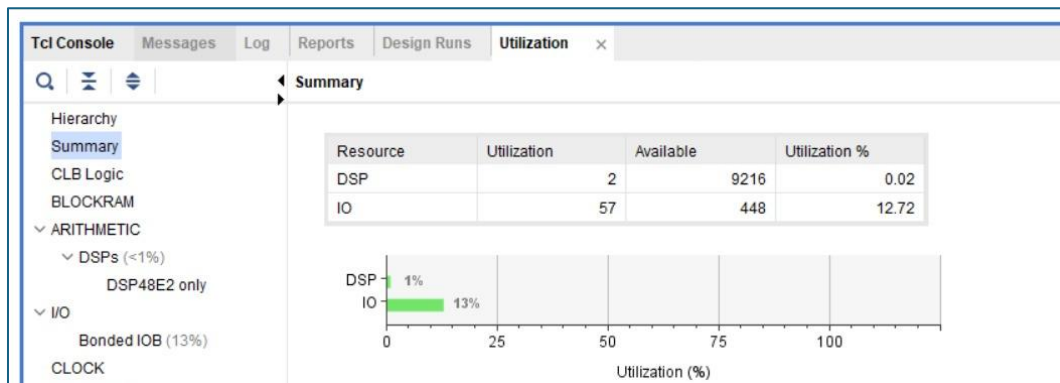


Figure 4: Multiplier IP Utilization

## Key Engineering Conclusions:

- **Multiplier Resource Allocation:** Synthesis results confirm that a 19-bit multiplication requires **2 DSP48 slices** to meet the 2.5 ns timing target. This allocation allows the IP to distribute the partial products and internal summation across two hardware blocks, preventing the negative slack often encountered in single-slice 19-bit designs at high frequencies.
- **Pipelining for Maximum Slack:** To isolate the critical path, we implemented **registers both before and after** the mult\_gen\_0 and c\_addsub\_0 blocks. This deep pipelining strategy minimizes combinational delays and ensures the system can scale to parallel units without timing violations.
- **Fabric vs. DSP Trade-off:** While LUT-based (fabric) implementation can theoretically reach a period of 1.25 ns, we have mandated the use of **DSP blocks only**. This preserves the FPGA fabric for routing and control logic, ensuring more predictable timing and lower power consumption for the massive parallel operations in Stage 1 and Stage 2.



## 1.2 Virtex UltraScale+ FPGA Feature Summary

|                            | VU3P    | VU5P      | VU7P      | VU9P      | VU11P     | VU13P     | VU19P     | VU23P     | VU27P     | VU29P     |
|----------------------------|---------|-----------|-----------|-----------|-----------|-----------|-----------|-----------|-----------|-----------|
| System Logic Cells         | 862,050 | 1,313,763 | 1,724,100 | 2,586,150 | 2,835,000 | 3,780,000 | 8,937,600 | 2,252,250 | 2,835,000 | 3,780,000 |
| CLB Flip-Flops             | 788,160 | 1,201,154 | 1,576,320 | 2,364,480 | 2,592,000 | 3,456,000 | 8,171,520 | 2,059,200 | 2,592,000 | 3,456,000 |
| CLB LUTs                   | 394,080 | 600,577   | 788,160   | 1,182,240 | 1,296,000 | 1,728,000 | 4,085,760 | 1,029,600 | 1,296,000 | 1,728,000 |
| Max. Distributed RAM (Mb)  | 12.0    | 18.3      | 24.1      | 36.1      | 36.2      | 48.3      | 58.4      | 14.2      | 36.2      | 48.3      |
| Block RAM Blocks           | 720     | 1,024     | 1,440     | 2,160     | 2,016     | 2,688     | 2,160     | 2,112     | 2,016     | 2,688     |
| Block RAM (Mb)             | 25.3    | 36.0      | 50.6      | 75.9      | 70.9      | 94.5      | 75.9      | 74.3      | 70.9      | 94.5      |
| UltraRAM Blocks            | 320     | 470       | 640       | 960       | 960       | 1,280     | 320       | 352       | 960       | 1,280     |
| UltraRAM (Mb)              | 90.0    | 132.2     | 180.0     | 270.0     | 270.0     | 360.0     | 90.0      | 99.0      | 270.0     | 360.0     |
| HBM DRAM (GB)              | -       | -         | -         | -         | -         | -         | -         | -         | -         | -         |
| CMTs (1 MMCM and 2 PLLs)   | 10      | 20        | 20        | 30        | 12        | 16        | 40        | 11        | 16        | 16        |
| Max. HP I/O <sup>(1)</sup> | 520     | 832       | 832       | 832       | 624       | 832       | 1,976     | 572       | 676       | 676       |
| Max. HD I/O <sup>(2)</sup> | 0       | 0         | 0         | 0         | 0         | 0         | 96        | 72        | 0         | 0         |
| DSP Slices                 | 2,280   | 3,474     | 4,560     | 6,840     | 9,216     | 12,288    | 3,840     | 1,320     | 9,216     | 12,288    |

Figure 5: Virtex UltraScale+ FPGA Resources

## 1.3 DSP48E2 Resource Mapping for 19-bit Arithmetic

To achieve high-precision multiplication and efficient accumulation, we utilize the DSP48E2 slices of the Virtex UltraScale+ architecture. While a single DSP slice is highly optimized, our 19-bit data width requires a specific multi-slice configuration to maintain accuracy and throughput.

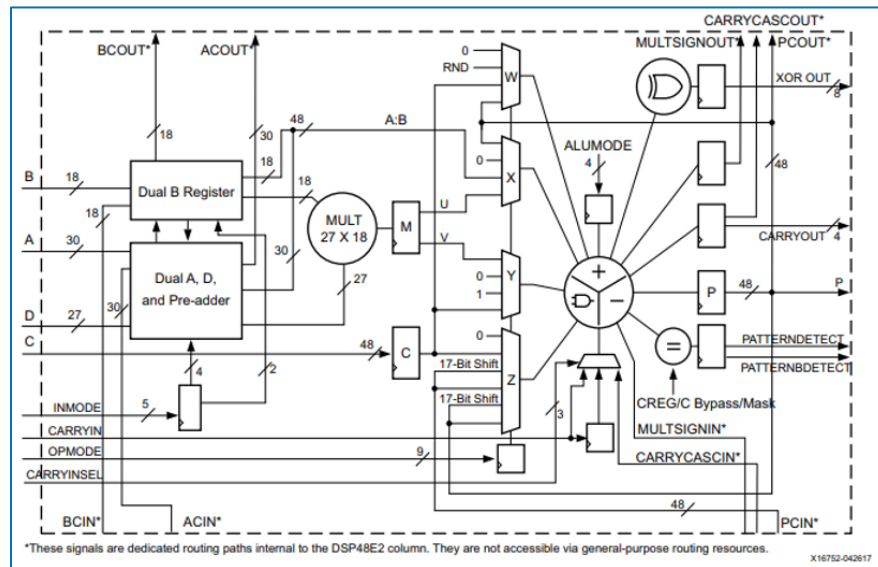


Figure 6: DSP48E2 Slice

### A. Multiplication: The Need for 2 DSP Slices

The native multiplier in a single DSP48E2 slice is designed for **27 x 18-bit** (signed) operations.

- **The Constraint:** Since our data elements are **19-bit**, they exceed the 18-bit limit of the 'B' input port.
- **The Solution:** To perform a full **19 x 19-bit** multiplication without losing precision, the synthesis tool combines **two DSP48E2 slices**. This allows the design to handle the extra bit width while maintaining the **400 MHz** clock frequency through dedicated cascading paths.



## B. Efficient Addition: SIMD Mode Capabilities

The DSP48E2 features a flexible **48-bit ALU (Adder/Subtractor)**. For our 19-bit elements, this large width provides a unique opportunity for optimization:

- **Width Advantage:** A 48-bit adder is significantly wider than our 19-bit requirements.
- **SIMD (Single Instruction, Multiple Data) Mode:** Because  $19 \times 2 = 38$  bits (which is less than 48), a single DSP slice can be configured to perform **two 19-bit additions simultaneously**.
- **Implementation:** By splitting the 48-bit carry chain, one DSP slice acts as a dual-adder. This effectively doubles our accumulation throughput for the Stage 1 and Stage 2 summation trees while reducing the total DSP count by half for the addition stages.

## 1.4 Timing Constraints & Critical Path Management

To guide the Vivado implementation tools and guarantee the measured performance, the following XDC (Xilinx Design Constraints) were established:

### Multiplier Constraint (mult\_gen\_0):

```
create_clock -period 2.500 -name mul_clk [get_ports clk]
```

### Adder Constraint (c\_addsub\_0):

```
create_clock -period 1.400 -name adder_clk [get_ports clk]
```

| Setup                                | Hold                             | Pulse Width                                       |
|--------------------------------------|----------------------------------|---------------------------------------------------|
| Worst Negative Slack (WNS): 0.105 ns | Worst Hold Slack (WHS): 0.213 ns | Worst Pulse Width Slack (WPWS): 0.278 ns          |
| Total Negative Slack (TNS): 0.000 ns | Total Hold Slack (THS): 0.000 ns | Total Pulse Width Negative Slack (TPWS): 0.000 ns |
| Number of Failing Endpoints: 0       | Number of Failing Endpoints: 0   | Number of Failing Endpoints: 0                    |
| Total Number of Endpoints: 19        | Total Number of Endpoints: 19    | Total Number of Endpoints: 58                     |

All user specified timing constraints are met.

Figure 7: Adder IP Timing Summary

### The Throughput Ratio

In the time it takes to complete **one multiplication cycle (2.5 ns)**, you have enough time to complete **nearly two addition cycles** sequentially:

- $2.5 \text{ ns} / 1.4 \text{ ns} \approx 1.78$  **Addition Windows**.

### The Practical Calculation

Because each DSP slice performs **two additions in parallel** (SIMD mode), the total "Addition Capacity" per multiplier cycle is:

- $1.78 \text{ (time windows)} \times 2 \text{ (parallel operations)} = 3.56$

In the time it takes to finish **one** multiplication cycle, the hardware has the capacity to perform **3 to 4 additions**.



## 1.5 Advanced Mathematical Optimization (SIMD Logic)

An essential discovery in our architecture is the ability to exploit the **48-bit width** of the DSP48E2 adder. Since our data elements are **19-bit**, each physical c\_addsub\_0 unit is capable of being optimized for dual-addition (SIMD mode).

- **The Conclusion:** We can effectively double our summation throughput by performing two 19-bit additions within a single 48-bit DSP adder ( $19 \times 2 = 38$  bits, which fits comfortably within the 48-bit limit).
- **Impact on System Design:** Based on this, we have revised our computational models to assume a throughput of **1408**  $\times$  **2** additions per cycle, significantly reducing the total cycle count for our Stage 1 selection and pooling processes.

## 2 BRAM Optimization and Memory Bandwidth Analysis

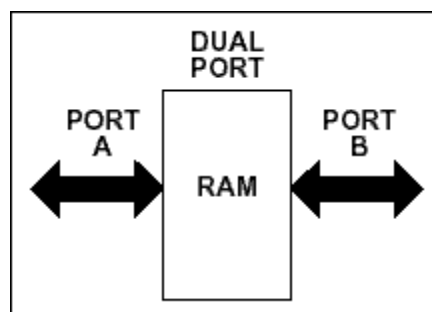
This section details the strategic utilization of the **70.9 Mb Block RAM** available on the **Virtex UltraScale+ XCVU11P**. Our analysis focused on maximizing storage density for our **19-bit** data elements and calculating the peak fetch bandwidth available to the parallel compute stages based on the native UltraScale architecture.

| Virtex UltraScale+ FPGA (VU11P) |         |
|---------------------------------|---------|
| Single BRAM                     | 36 Kb   |
| Block RAM Blocks                | 2,016   |
| Block RAM                       | 70.9 Mb |

### 2.1 Architectural Features of UltraScale+ BRAM

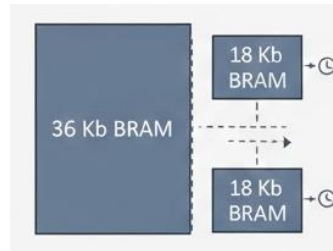
According to the UltraScale architecture specifications, the memory subsystem is built upon highly flexible **36 Kb Block RAM** tiles. Key features relevant to our 19-bit accelerator include:

- **Dual-Port Flexibility:** Each block RAM features two completely independent ports sharing only the stored data. These ports can have **different aspect ratios** without constraints, allowing for specialized read/write configurations.

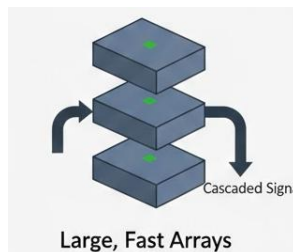




- **18 Kb Partitioning:** Each 36 Kb block can be divided into two completely independent **18 Kb block RAMs**. Each 18 Kb unit can be configured in various aspect ratios (from  $16K \times 1$  to  $512 \times 36$ ).



- **Registration and Pipelining:** All inputs (data, address, enables) are registered. We have opted to enable the **optional output data pipeline register**, which allows for higher clock rates (matching our 2.5 ns target) at the cost of one extra cycle of latency.
- **Vertical Cascading:** Connections within block RAM columns allow signals to be **cascaded between vertically adjacent blocks**. This enables the creation of large, fast memory arrays for our feature maps with significantly reduced power consumption compared to using fabric logic.



## 2.2 Native Configurations and Parity Integration

The native configurations of the 36 Kb block RAM allow us to leverage "parity" bits for functional data storage, which is critical for our 19-bit elements.

### Native Configurations of a 36 Kb BRAM Tile

| Configuration Mode | Depth (Addresses) | Width (Bits) | Total Data Bits | Parity Bits Used | Total Bits |
|--------------------|-------------------|--------------|-----------------|------------------|------------|
| 32K x 1            | 32,768            | 1            | 32,768          | 0                | 32,768     |
| 4K x 9             | 4,096             | 9            | 32,768          | 4,096            | 36,864     |
| 2K x 18            | 2,048             | 18           | 32,768          | 4,096            | 36,864     |
| 1K x 36            | 1,024             | 36           | 32,768          | 4,096            | 36,864     |
| 512 x 72 (SDP)     | 512               | 72           | 65,536          | 8,192            | 73,728     |





## 2.3 Design Trade-off Analysis: 19-bit Element Mapping

Since data widths greater than 36 bits (for 36 Kb RAM) can only be accessed in **Simple Dual-Port (SDP) mode**, we evaluated two strategies:

### Strategy A: Standard Dual-Port Mapping (36-bit Width)

- **Configuration:** 1,024 Depth × 36-bit Width.
- **Mechanism:** One 19-bit element per address.
- **Performance:** Provides two independent read ports, but results in **47.2% waste** of the physical BRAM capacity (17 unused bits per row).

### Strategy B: Optimized SDP Concatenation (72-bit Width)

- **Configuration:** 512 Depth × 72-bit Width (Simple Dual-Port Mode).
- **Mechanism:** One port is dedicated to reading, the other to writing. We pack **three 19-bit elements** side-by-side ( $19 \times 3 = 57$  bits) into the 72-bit wide interface.
- **Performance:** This utilizes the parity bits effectively and minimizes waste (only 15 bits unused). While it restricts us to one dedicated read port per block, the high width allows us to fetch **three elements simultaneously**.

## 2.4 Bandwidth and Throughput Conclusion

To justify our final architectural choice, we conducted a throughput analysis comparing the two storage strategies using the allocated **1,408 BRAM blocks**. This comparison demonstrates how the physical configuration of the memory directly impacts the computational efficiency of the accelerator.

### Comparative Throughput Metrics

| Metric                   | Strategy A: Standard (36-bit) | Strategy B: Optimized (72-bit SDP) |
|--------------------------|-------------------------------|------------------------------------|
| Utilized BRAM Blocks     | 1,408                         | 1,408                              |
| Elements per Read Port   | 1 Element (19-bit)            | <b>3 Elements (19-bit packed)</b>  |
| Active Ports per Block   | 2 (True Dual-Port)            | 1 (Simple Dual-Port)               |
| Total Elements per Cycle | 2,816 Elements                | <b>4,224 Elements</b>              |
| BRAM Storage Efficiency  | ~52.8% (47.2% Waste)          | <b>~79.2% (Minimal Waste)</b>      |



## 2.5 Storage Feasibility

A critical requirement is the ability to store the output of the convolution layer, which has a dimension of **256 x 256 x 32**.

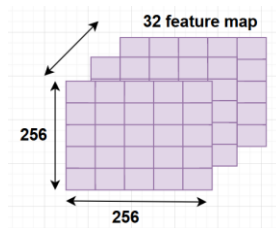
### Required Storage Calculation:

- **Total Elements:**  $256 \times 256 \times 32 = 2,097,152$  elements.
- **Bit Requirements:**  $2,097,152 \times 19 \text{ bits} \approx 39.85 \text{ Mb}$ .

### Available Capacity (1,408 Blocks):

- **Physical Capacity:**  $1,408 \times 36,864 \text{ bits} \approx 51.9 \text{ Mb}$ .
- **Effective Capacity (Strategy B):** Accounting for the 57-bit/72-bit packing ratio  $(57/72 \times 51.9)$ : 41.1 Mb.

| Parameter           | Required Size | Effective Capacity | Result     |
|---------------------|---------------|--------------------|------------|
| Feature Map Storage | 39.85 Mb      | 41.10 Mb           | Sufficient |



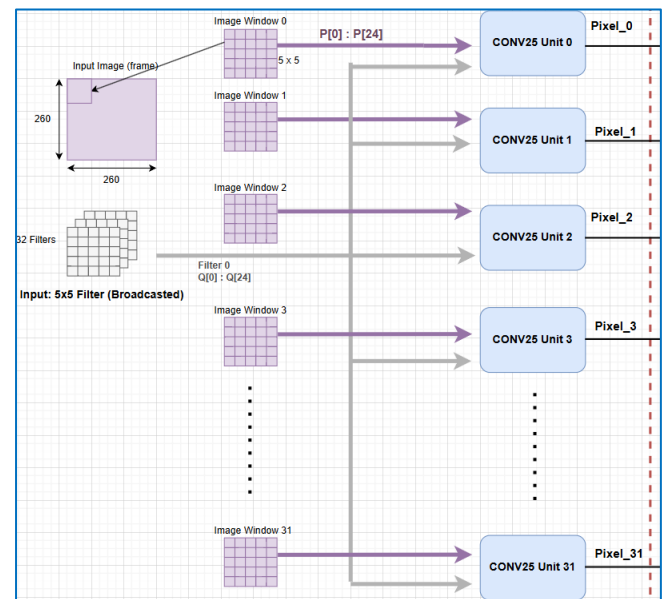
## 2.6 Computational Bandwidth Alignment: Stage 1 Case Study

To validate the effectiveness of our **72-bit SDP Concatenation Strategy**, we analyzed the data requirements for the Stage 1 Convolution layer. This case study demonstrates how the optimized memory bandwidth directly supports our massive parallel processing requirements.

### Scenario: Parallel 5x5 Convolution

In Stage 1, the architecture processes **32 image windows** simultaneously. Each window is a  $5 \times 5$  sub-grid assigned to a dedicated CONV25 unit.

- **Image Data:** 32 windows  $\times$  25 pixels/window = **800 elements**.
- **Filter Data:** 1 common  $5 \times 5$  filter broadcasted to all units = **25 elements**.
- **Total Operands Required:** 825 elements per operational cycle.



## Throughput Performance vs. Requirement



Using the parameters established in the previous sections, we compare the operand requirement against the capability of our **1,408 BRAM blocks**:

| Parameter                        | Value                |
|----------------------------------|----------------------|
| Peak Fetch Capacity (Strategy B) | 4,224 elements/cycle |
| Stage 1 Operand Requirement      | 825 elements/cycle   |
| Bandwidth Utilization            | 19.5%                |
| Operational Headroom             | ~5.1x                |